



Stacks

1. Introduction to Stacks

A *stack* is a linear data structure that follows the *Last In, First Out (LIFO)* principle. This means the last element added to the stack will be the first one to be removed.

Properties of Stack:

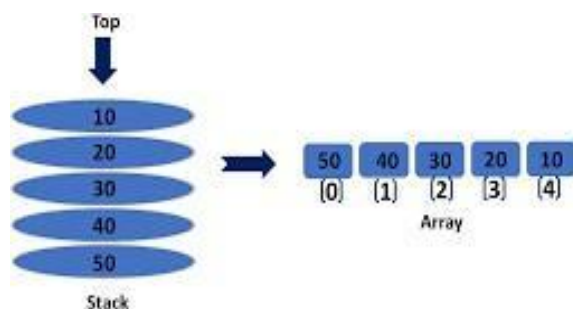
- **LIFO** structure.
- Only one end (top) is accessible for adding (push) and removing (pop) elements.
- Can be implemented using **arrays** or **linked lists**.

Basic Stack Operations:

- **push()**: Adds an element to the top of the stack.
- **pop()**: Removes the top element from the stack.
- **peek() / top()**: Returns the top element without removing it.
- **isEmpty()**: Checks whether the stack is empty.
- **isFull()**: Checks whether the stack is full (for array-based stacks).

Concept Explanation

A stack can be visualized as a stack of plates. You can only add (push) or remove (pop) the plate from the top of the stack. **The last plate added will be the first one removed**, which illustrates the LIFO principle.



Real-life Example:

Imagine a news website where the latest articles are added to a stack in reverse chronological order (newest article on top). Users can interact with this stack to view or remove articles.

1. STACK IMPLEMENTATION

1.1 Stack Using Array

Code Implementation:

```
#Stack using array
MAX = 6
stack = [None] * MAX
top = -1

def push(value):
    global top
    if top == MAX-1:
        print("Stack is full")
    else:
        top += 1
        stack[top] = value
        print("Pushed: ",value)

def pop():
    global top
    if top == -1:
        print("Stack is empty cann't pop")
    else:
        p = stack[top]
        top -= 1
        print("Popped :",p)

def peek():
    if top == -1:
        print("Stack is empty")
    else:
        print("Top element",stack[top])

def is_empty():
    return top == -1

def is_full():
    return top == MAX - 1

def display():
    if is_empty():
        print("Stack is empty")
    else:
```

```

    print("Stack elements")
    for i in range(top, -1, -1):
        print(stack[i])

# Testing the stack operations
push(10)
push(20)
push(30)
push(40)
push(50)
display()
peek()
pop()
display()

```

Output:

```

Pushed: 10
Pushed: 20
Pushed: 30
Pushed: 40
Pushed: 50
Stack elements
50
40
30
20
10
Top element 50
Popped : 50
Stack elements
40
30
20
10

```

Step-by-Step Process:

Initialization:

- **MAX = 6**: Maximum size of stack
- **stack = [None, None, None, None, None, None]**: Array initialized with None values
- **top = -1**: Points to the top element (initially empty)

Step 1: push(10)

- Check if stack is full: **top == MAX-1** → **-1 == 5** → False
- Increment top: **top = 0**
- Insert value: **stack[0] = 10**
- Stack state: **[10, None, None, None, None, None]**, top = 0

Step 2: push(20)

- Check if stack is full: $top == MAX-1 \rightarrow 0 == 5 \rightarrow \text{False}$
- Increment top: $top = 1$
- Insert value: $stack[1] = 20$
- Stack state: $[10, 20, \text{None}, \text{None}, \text{None}, \text{None}]$, $top = 1$

Step 3: push(30)

- Check if stack is full: $top == MAX-1 \rightarrow 1 == 5 \rightarrow \text{False}$
- Increment top: $top = 2$
- Insert value: $stack[2] = 30$
- Stack state: $[10, 20, 30, \text{None}, \text{None}, \text{None}]$, $top = 2$

Step 4: push(40)

- Check if stack is full: $top == MAX-1 \rightarrow 2 == 5 \rightarrow \text{False}$
- Increment top: $top = 3$
- Insert value: $stack[3] = 40$
- Stack state: $[10, 20, 30, 40, \text{None}, \text{None}]$, $top = 3$

Step 5: push(50)

- Check if stack is full: $top == MAX-1 \rightarrow 3 == 5 \rightarrow \text{False}$
- Increment top: $top = 4$
- Insert value: $stack[4] = 50$
- Stack state: $[10, 20, 30, 40, 50, \text{None}]$, $top = 4$

Step 6: display()

- Check if empty: $top == -1 \rightarrow 4 == -1 \rightarrow \text{False}$
- Print elements from top to bottom: 50, 40, 30, 20, 10

Step 7: peek()

- Check if empty: $top == -1 \rightarrow 4 == -1 \rightarrow \text{False}$
- Print top element: $stack[4] = 50$

Step 8: pop()

- Check if empty: $top == -1 \rightarrow 4 == -1 \rightarrow \text{False}$
- Store popped value: $p = stack[4] = 50$
- Decrement top: $top = 3$
- Stack state: $[10, 20, 30, 40, 50, \text{None}]$, $top = 3$ (50 is logically removed)

Step 9: display()

- Print elements from top to bottom: 40, 30, 20, 10

1.2 Stack Using Linked List

Code Implementation:

```
#Stack using linked list
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class stack:
    def __init__(self):
        self.top = None

    def is_empty(self):
        return self.top == None

    def push(self, data):
        newnode = Node(data)
        newnode.next = self.top
        self.top = newnode

    def pop(self):
        if self.is_empty():
            print("Stack is empty can't pop")
        else:
            p = self.top.data
            self.top = self.top.next
            print("Popped :", p)

    def peek(self):
        if self.is_empty():
            print("Stack is empty")
        else:
            print("Peek :", self.top.data)

    def display(self):
        if self.is_empty():
            print("Stack is empty")
        else:
            n = self.top
            while(n):
                print(n.data)
                n = n.next

# Testing the stack operations
s = stack()
s.push(10)
s.push(20)
s.push(30)
s.display()
s.peek()
```

```
s.pop()
s.display()
```

Output:

```
10
20
30
Peek : 30
Popped : 20
10
30
```

Step-by-Step Process:**Initialization:**

- Create stack object `s` with `top = None`

Step 1: s.push(10)

- Create new node: `Node(10)` with data=10, next=None
- Set `newnode.next = self.top` → `newnode.next = None`
- Set `self.top = newnode`
- Stack: `top` → `[10|None]`

Step 2: s.push(20)

- Create new node: `Node(20)` with data=20, next=None
- Set `newnode.next = self.top` → points to node with data=10
- Set `self.top = newnode`
- Stack: `top` → `[20|→]` → `[10|None]`

Step 3: s.push(30)

- Create new node: `Node(30)` with data=30, next=None
- Set `newnode.next = self.top` → points to node with data=20
- Set `self.top = newnode`
- Stack: `top` → `[30|→]` → `[20|→]` → `[10|None]`

Step 4: s.display()

- Start from top: print 30, 20, 10

Step 5: s.peek()

- Print top element: `self.top.data = 30`

Step 6: s.pop()

- Store popped value: `p = self.top.data = 30`
- Move top to next: `self.top = self.top.next`
- Stack: `top → [20|→] → [10|None]`
- Print "Popped : 30"

Step 7: s.display()

- Start from top: print 20, 10
-



Queues

Queues – Introduction, and Implementation

1. Introduction

A **queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle. It means that the element inserted first will be the first to be removed. This concept is widely used in real- life scenarios like a line at a ticket counter or customer service desk, where the first person to join the line is the first to be served.

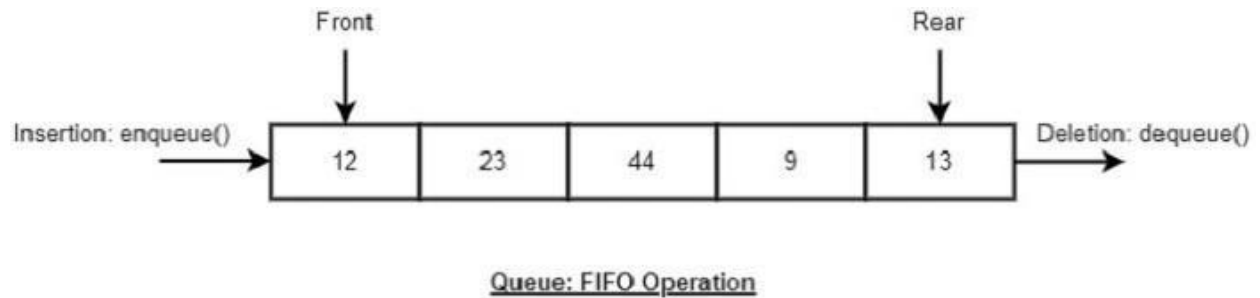
Queues are essential in computer science and are used in scheduling, buffer management, and many other applications where order needs to be preserved.

Queue Operations Using Arrays:

In an array-based queue, we maintain two pointers:

- **Front:** Points to the first element in the queue.
- **Rear:** Points to the last element in the queue.

Let's look at the basic operations in C:



Main Operations Supported by a Queue:

1. **Enqueue:**
 - o Adds (inserts) an element to the rear (end) of the queue.
2. **Dequeue:**
 - o Removes and returns the element from the front of the queue. If the queue is empty, this operation usually causes an underflow error.
3. **Front (or Peek):**
 - o Returns the element at the front of the queue without removing it.
4. **isEmpty:**
 - o Checks whether the queue is empty.
5. **isFull** (in case of array implementation):
 - o Checks whether the queue is full (for fixed-size implementations).

Different Ways of Implementing a Queue:

1. **Using Arrays:**
 - o A fixed-size queue is implemented using an array.
 - o Simple to implement but may lead to inefficient use of space (solved by circular queue).

2. QUEUE IMPLEMENTATION

2.1 Queue Using Array

Code Implementation:

```
#Queue using array
MAX = 4
queue = [None] * MAX
front = -1
rear = -1
```

```

def enqueue(val):
    global front, rear
    if rear == MAX-1:
        print("Queue is full")
        return
    if front == -1:
        front = 0
    rear += 1
    queue[rear] = val

def dequeue():
    global front, rear
    if front == -1 or front > rear:
        print("Queue is empty")
    else:
        val = queue[front]
        front += 1
        print("\ndeleted from front :", val)

def display():
    if front == -1 or front > rear:
        print("Queue is empty")
    else:
        print("Queue")
        for i in range(front, rear+1):
            print(queue[i], end=" ")

# Testing the queue operations
enqueue(10)
enqueue(20)
enqueue(30)
display()
dequeue()
display()

```

Output:

```

Queue
10 20 30
deleted from front : 10
Queue
20 30

```

Step-by-Step Process:

Initialization:

- **MAX = 4**: Maximum size of queue
- **queue = [None, None, None, None]**: Array initialized
- **front = -1, rear = -1**: Both pointers initially empty

Step 1: enqueue(10)

- Check if full: `rear == MAX-1` → `-1 == 3` → False
- Set front: `front = 0` (since front was -1)
- Increment rear: `rear = 0`
- Insert value: `queue[0] = 10`
- Queue state: `[10, None, None, None]`, front=0, rear=0

Step 2: enqueue(20)

- Check if full: `rear == MAX-1` → `0 == 3` → False
- Front already set, so skip
- Increment rear: `rear = 1`
- Insert value: `queue[1] = 20`
- Queue state: `[10, 20, None, None]`, front=0, rear=1

Step 3: enqueue(30)

- Check if full: `rear == MAX-1` → `1 == 3` → False
- Front already set, so skip
- Increment rear: `rear = 2`
- Insert value: `queue[2] = 30`
- Queue state: `[10, 20, 30, None]`, front=0, rear=2

Step 4: display()

- Check if empty: `front == -1 or front > rear` → `0 == -1 or 0 > 2` → False
- Print elements from front to rear: 10, 20, 30

Step 5: dequeue()

- Check if empty: `front == -1 or front > rear` → `0 == -1 or 0 > 2` → False
- Store dequeued value: `val = queue[0] = 10`
- Increment front: `front = 1`
- Queue state: `[10, 20, 30, None]`, front=1, rear=2 (10 is logically removed)

Step 6: display()

- Print elements from front to rear: 20, 30

2.2 Queue Using Linked List

Code Implementation:

#Queue using linked list

class Node:

```
def __init__(self, data):  
    self.data = data
```

```

        self.next = None

front = None
rear = None

def enqueue(val):
    global front, rear
    newnode = Node(val)
    if rear is None:
        front = rear = newnode
    else:
        rear.next = newnode
        rear = newnode

def dequeue():
    global front, rear
    if front == None:
        print("Queue is empty")
    front = front.next

def display():
    n = front
    print("\nQueue")
    while(n):
        print(n.data, end=" ")
        n = n.next

# Testing the queue operations
enqueue(10)
enqueue(20)
enqueue(30)
display()
dequeue()
display()

```

Output:

```

Queue
10 20 30
Queue
20 30

```

Step-by-Step Process:

Initialization:

- `front = None, rear = None`: Both pointers empty

Step 1: enqueue(10)

- Create new node: `Node(10)` with data=10, next=None

- Since rear is None: `front = rear = newnode`
- Queue: `front → [10|None] ← rear`

Step 2: enqueue(20)

- Create new node: `Node(20)` with data=20, next=None
- Since rear is not None: `rear.next = newnode, rear = newnode`
- Queue: `front → [10|→] → [20|None] ← rear`

Step 3: enqueue(30)

- Create new node: `Node(30)` with data=30, next=None
- Since rear is not None: `rear.next = newnode, rear = newnode`
- Queue: `front → [10|→] → [20|→] → [30|None] ← rear`

Step 4: display()

- Start from front: print 10, 20, 30

Step 5: dequeue()

- Check if empty: `front == None → False`
- Move front to next: `front = front.next`
- Queue: `front → [20|→] → [30|None] ← rear`

Step 6: display()

- Start from front: print 20, 30

Time Complexities

- **Stack Operations:** $O(1)$ for push, pop, peek
- **Queue Operations:** $O(1)$ for enqueue, dequeue
- **Space Complexity:** $O(n)$ where n is the number of elements

3. CIRCULAR QUEUE IMPLEMENTATION

3.1 Circular Queue Using Array

Code Implementation:

```
#Circular Queue using array
size = 4
```

```

queue = [None] * size
front = -1
rear = -1

def enqueue(val):
    global front, rear
    if (front == 0 and rear == size-1) or ((rear+1) % size == front):
        print("Circular queue is full")
    else:
        if front == -1:
            front = 0
        rear = (rear+1) % size
        queue[rear] = val
        print(val, " Value added in index ", rear)

def dequeue():
    global front, rear
    if front == -1:
        print("Circular queue is empty")
        return
    print("\nDeleted queue :", queue[front])
    if front == rear:
        front = rear = -1
    else:
        front = (front + 1) % size

def display():
    if front == -1:
        print("Circular queue is empty")
    i = front
    while True:
        print(queue[i], end=" ")
        if i == rear:
            break
        i = (i+1) % size

# Testing the circular queue operations
enqueue(10)
enqueue(20)
enqueue(30)
enqueue(40)
display()
dequeue()
enqueue(50)
display()

```

Output:

```

10 Value added in index 0
20 Value added in index 1
30 Value added in index 2
40 Value added in index 3
10 20 30 40

```

Deleted queue : 10
50 Value added in index 0
20 30 40 50

Step-by-Step Process:

Initialization:

- `size = 4`: Maximum size of circular queue
- `queue = [None, None, None, None]`: Array initialized
- `front = -1, rear = -1`: Both pointers initially empty

Step 1: enqueue(10)

- Check if full: $(front == 0 \text{ and } rear == size-1) \text{ or } ((rear+1) \% size == front) \rightarrow \text{False}$
- Set front: `front = 0` (since front was -1)
- Calculate rear: `rear = (rear+1) \% size = ((-1)+1) \% 4 = 0`
- Insert value: `queue[0] = 10`
- Queue state: `[10, None, None, None]`, front=0, rear=0

Step 2: enqueue(20)

- Check if full: $(front == 0 \text{ and } rear == size-1) \text{ or } ((rear+1) \% size == front) \rightarrow (0 == 0 \text{ and } 0 == 3) \text{ or } ((0+1) \% 4 == 0) \rightarrow \text{False or } (1 == 0) \rightarrow \text{False}$
- Front already set
- Calculate rear: `rear = (rear+1) \% size = (0+1) \% 4 = 1`
- Insert value: `queue[1] = 20`
- Queue state: `[10, 20, None, None]`, front=0, rear=1

Step 3: enqueue(30)

- Check if full: condition evaluates to False
- Calculate rear: `rear = (rear+1) \% size = (1+1) \% 4 = 2`
- Insert value: `queue[2] = 30`
- Queue state: `[10, 20, 30, None]`, front=0, rear=2

Step 4: enqueue(40)

- Check if full: condition evaluates to False
- Calculate rear: `rear = (rear+1) \% size = (2+1) \% 4 = 3`
- Insert value: `queue[3] = 40`
- Queue state: `[10, 20, 30, 40]`, front=0, rear=3

Step 5: display()

- Start from front=0, display until rear=3: 10, 20, 30, 40

Step 6: dequeue()

- Check if empty: $\text{front} == -1 \rightarrow 0 == -1 \rightarrow \text{False}$
- Store dequeued value: $\text{queue}[0] = 10$
- Check if single element: $\text{front} == \text{rear} \rightarrow 0 == 3 \rightarrow \text{False}$
- Calculate new front: $\text{front} = (\text{front} + 1) \% \text{size} = (0+1) \% 4 = 1$
- Queue state: $[10, 20, 30, 40]$, front=1, rear=3 (10 is logically removed)

Step 7: enqueue(50)

- Check if full: condition evaluates to False
- Calculate rear: $\text{rear} = (\text{rear}+1) \% \text{size} = (3+1) \% 4 = 0$ (wraps around)
- Insert value: $\text{queue}[0] = 50$ (reuses the space)
- Queue state: $[50, 20, 30, 40]$, front=1, rear=0

Step 8: display()

- Start from front=1: 20, 30, 40, then wrap to index 0: 50
 - Output: 20, 30, 40, 50
-

1. Balanced Parentheses Checker

Program Name

Check Balancing of Parentheses using Stack

Algorithm

1. Initialize an empty stack
2. Traverse the expression character by character
3. If character is opening bracket ('(', '{', '['), push to stack
4. If character is closing bracket (')', '}', ']'):
 - Pop from stack
 - Check if popped character matches current closing bracket
 - If no match, expression is unbalanced
5. After traversal, if stack is empty, expression is balanced

2. Postfix Expression Evaluator

Program Name

Evaluate Postfix Expression using Stack

Algorithm

1. Initialize an empty stack
2. Scan postfix expression from left to right
3. If element is operand, push to stack
4. If element is operator:
 - Pop two operands from stack
 - Apply operator (second_operand operator first_operand)
 - Push result back to stack
5. Final result is the only element left in stack

3. Array Reversal Using Stack

Program Name

Reverse an Array Using Stack

Algorithm

1. Create a stack
2. Push all array elements onto the stack
3. Pop elements from stack back into the array
4. Due to LIFO nature, array gets reversed

4. Infix to Postfix Conversion

Program Name

Convert Infix Expression to Postfix using Stack

Algorithm

1. Initialize empty stack and result string
2. Scan infix expression from left to right
3. If operand, add to result
4. If '(', push to stack
5. If ')', pop until '(' and add to result
6. If operator:
 - Pop operators with higher/equal precedence
 - Push current operator
7. Pop remaining operators from stack

5. Palindrome Checker Using Stack

Program Name

Check if String is Palindrome using Stack

Algorithm

1. Push all characters of string onto stack
2. Compare each character of string with popped character from stack
3. If all characters match, string is palindrome
4. If any character doesn't match, string is not palindrome

6. Symmetry Checker Using Stack and Queue

Program Name

Check String Symmetry using Stack and Queue

Algorithm

1. Initialize stack and queue
2. Add all characters to both stack and queue
3. Compare characters popped from stack with characters from queue

4. If all match, string is symmetric (palindrome)
5. If any don't match, string is not symmetric